# Table of Contents

Unit II — Overview of Object Oriented Programming using Java

- 2.1 Introduction to OOP and Java
- 2.2 Inheritance
- 2.3 Polymorphism
- 2.4 Interfaces
- 2.5 Abstract Classes
- 2.6 Threads
- 2.7 Method Overloading and Overriding
- 2.8 Java Virtual Machine (JVM)
- 2.9 Unit II Summary & Practice Questions

Unit III — Development Tools

- 3.1 Introduction to Android Development Environment
- 3.2 Installing and Using Eclipse with the ADT Plug-in
- 3.3 Installing a Virtual Machine for Android (Emulator)
- 3.4 Configuring the Installed Tools
- 3.5 Creating an Android Project
- 3.6 Hello World — Running on the Emulator
- 3.7 Deploying on a USB-Connected Android Device
- 3.8 Unit III Summary & Practice Questions

# UNIT II

***Overview of Object Oriented Programming using Java***

## 2.1 Introduction to OOP and Java

Object Oriented Programming (OOP) is a programming paradigm built around the idea of "objects" — self-contained units that bundle data (fields/attributes) with the behavior (methods) that acts on that data. Java is a purely object-oriented language (except for primitive types) and implements OOP through four foundational pillars:

- Encapsulation — binding data and methods together, restricting direct access via access modifiers (private, protected, public)
- Inheritance — acquiring properties and behavior of one class into another
- Polymorphism — the ability of an object/method to take many forms
- Abstraction — hiding implementation detail and exposing only essential features

This unit covers Inheritance, Polymorphism, Interfaces, Abstract Classes, Threads, and Overloading/Overriding — the mechanisms Java provides to implement these pillars — followed by the Java Virtual Machine, which is what allows Java's "write once, run anywhere" promise to work.

## 2.2 Inheritance

Inheritance is the mechanism by which one class (subclass/child class) acquires the fields and methods of another class (superclass/parent class), using the extends keyword. It models an "IS-A" relationship and promotes code reusability.

### Types of Inheritance in Java

| Type | Description | Supported Directly in Java? |
|---|---|---|
| Single | One subclass extends one superclass | Yes |
| Multilevel | A class is derived from a class which is itself derived from another class | Yes |
| Hierarchical | Multiple subclasses extend a single superclass | Yes |
| Multiple | One class extends more than one class directly | No (achieved via interfaces) |
| Hybrid | Combination of two or more inheritance types | Via interfaces only |

> **Note:** Java does not support multiple inheritance of classes to avoid the "Diamond Problem" (ambiguity when two parent classes have a method with the same signature). Multiple inheritance of type is achieved through interfaces instead.

### Example — Single & Multilevel Inheritance

```
class Animal {
    void eat() {
        System.out.println("This animal eats food.");
    }
}

class Dog extends Animal {          // single inheritance
    void bark() {
```

```
        System.out.println("The dog barks.");
    }
}

class Puppy extends Dog {           // multilevel inheritance
    void weep() {
        System.out.println("The puppy weeps.");
    }
}

public class TestInheritance {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat();     // inherited from Animal
        p.bark();    // inherited from Dog
        p.weep();    // defined in Puppy
    }
}
```

### The super Keyword

The super keyword refers to the immediate parent class object. It is used to:

- Call the parent class's constructor — super(args)
- Access the parent class's fields when hidden by the subclass
- Call the parent class's overridden method — super.methodName()

## 2.3 Polymorphism

Polymorphism (Greek: “many forms”) allows a single action, method name, or reference to behave differently depending on the context. Java supports two kinds:

| Type | Also Known As | Resolved At | Achieved Through |
|---|---|---|---|
| Compile-time Polymorphism | Static / Early Binding | Compile time | Method Overloading |
| Runtime Polymorphism | Dynamic / Late Binding | Run time | Method Overriding |

### Runtime Polymorphism Example

```
class Shape {
    void draw() {
        System.out.println("Drawing a generic shape");
    }
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a circle");
    }
}

class Square extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a square");
    }
}
```

```
public class TestPolymorphism {
    public static void main(String[] args) {
        Shape s;                // reference type is Shape
        s = new Circle();
        s.draw();               // "Drawing a circle" – decided at RUNTIME
        s = new Square();
        s.draw();               // "Drawing a square"
    }
}
```

Here, the same reference variable s and the same method call s.draw() produce different behavior depending on the actual object assigned at runtime — this is dynamic method dispatch, the core mechanism behind runtime polymorphism.

## 2.4 Interfaces

An interface in Java is a fully abstract type that specifies a contract of behavior — what a class must do, without saying how. It is declared using the interface keyword. Prior to Java 8, all interface methods were implicitly public and abstract; Java 8+ also allows default and static methods with bodies, and Java 9+ allows private methods inside interfaces.

### Key Characteristics

- A class implements an interface using the implements keyword
- A class can implement multiple interfaces — this is how Java achieves multiple inheritance of type
- All fields in an interface are implicitly public static final (constants)
- An interface can extend one or more other interfaces

### Example

```
interface Vehicle {
    void start();               // implicitly public abstract
    void stop();
}

interface Electric {
    void chargeBattery();
}

// multiple inheritance of type via interfaces
class ElectricCar implements Vehicle, Electric {
    public void start() {
        System.out.println("Car started silently.");
    }
    public void stop() {
        System.out.println("Car stopped.");
    }
    public void chargeBattery() {
        System.out.println("Battery charging...");
    }
}
```

### Abstract Class vs Interface

| Aspect | Abstract Class | Interface |
|---|---|---|
| Keyword | abstract class | interface |

| Aspect | Abstract Class | Interface |
|---|---|---|
| Methods | Can have abstract AND concrete methods | Traditionally all abstract; default/static allowed from Java 8+ |
| Variables | Can have any type of variable (instance, static, final) | Only public static final (constants) |
| Inheritance | A class can extend only ONE abstract class | A class can implement MULTIPLE interfaces |
| Constructors | Can have constructors | Cannot have constructors |
| Access Modifiers | Can use private, protected, public | Members are public by default |
| When to use | Related classes sharing common code/state | Unrelated classes sharing a common capability/contract |

## 2.5 Abstract Classes

An abstract class is a class declared with the abstract keyword that cannot be instantiated directly. It may contain a mix of fully implemented (concrete) methods and abstract methods (declared without a body, ending in a semicolon), which any concrete subclass is forced to implement.

### Example

```
abstract class Bank {
    abstract int getInterestRate();     // abstract method — no body

    void displayInterestRate() {        // concrete method
        System.out.println("Rate of Interest: " + getInterestRate() + "%");
    }
}

class SBI extends Bank {
    int getInterestRate() {
        return 7;
    }
}

class HDFC extends Bank {
    int getInterestRate() {
        return 8;
    }
}

public class TestBank {
    public static void main(String[] args) {
        Bank b1 = new SBI();
        b1.displayInterestRate();   // Rate of Interest: 7%
        Bank b2 = new HDFC();
        b2.displayInterestRate();   // Rate of Interest: 8%
    }
}
```

**Note:** An abstract class with zero concrete methods is functionally similar to an interface, but it can still hold instance state (non-final fields) and constructors — an interface cannot.

## 2.6 Threads

A thread is the smallest unit of execution within a process. Java supports multithreading — running multiple threads concurrently within a single program — through the java.lang.Thread class and the Runnable interface. Multithreading enables better CPU utilization and responsiveness, especially for I/O-bound or long-running tasks.

### Two Ways to Create a Thread

| Approach | How | Limitation |
|---|---|---|
| Extending Thread class | Override run(); call start() | Class cannot extend any other class (single inheritance) |
| Implementing Runnable interface | Implement run(); pass instance to a Thread object; call start() | None — preferred approach, since the class remains free to extend another class |

### Example — Implementing Runnable (preferred)

```
class MyTask implements Runnable {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(Thread.currentThread().getName() + " -> " + i);
        }
    }
}

public class TestThread {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyTask(), "Thread-A");
        Thread t2 = new Thread(new MyTask(), "Thread-B");
        t1.start();      // NOT t1.run() — start() creates a new call stack
        t2.start();
    }
}
```

### Thread Life Cycle

A thread moves through five states during its lifetime:

1. New — thread object created, but start() not yet called
2. Runnable — after start() is called; ready to run, waiting for CPU
3. Running — the thread scheduler has selected it for execution
4. Blocked / Waiting — temporarily inactive (waiting for a resource, or sleep()/wait() called)
5. Terminated (Dead) — the run() method has completed execution

## 2.7 Method Overloading and Overriding

### Method Overloading (Compile-time Polymorphism)

Overloading occurs when two or more methods in the same class share the same name but differ in the number, type, or order of parameters. The return type alone is not sufficient to overload a method. It is resolved by the compiler at compile time.

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
```

```
    double add(double a, double b) {           // different parameter type
        return a + b;
    }
    int add(int a, int b, int c) {    // different parameter count
        return a + b + c;
    }
}
```

### Method Overriding (Runtime Polymorphism)

Overriding occurs when a subclass provides its own specific implementation of a method that is already defined in its superclass, with the SAME name, SAME parameters, and a compatible return type. It is resolved at runtime based on the actual object type. The @Override annotation is optional but strongly recommended, since it lets the compiler flag mistakes such as a mismatched signature.

```
class Employee {
    void work() {
        System.out.println("Employee is working.");
    }
}

class Developer extends Employee {
    @Override
    void work() {
        System.out.println("Developer is writing code.");
    }
}
```

### Overloading vs Overriding — Quick Comparison

| Aspect | Overloading | Overriding |
|---|---|---|
| Relationship | Same class | Superclass–Subclass (inheritance) |
| Parameters | Must differ | Must be exactly the same |
| Return Type | Can differ | Same, or a covariant subtype |
| Binding | Static (compile time) | Dynamic (runtime) |
| Also Called | Compile-time polymorphism | Runtime polymorphism |
| Access Modifier | Can be anything | Cannot be more restrictive than the parent's method |

## 2.8 Java Virtual Machine (JVM)

The JVM is an abstract computing machine that provides a runtime environment in which Java bytecode (.class files) can execute. It is what makes Java platform-independent: the same compiled bytecode can run on any device that has a JVM, regardless of the underlying operating system or hardware — the famous “Write Once, Run Anywhere” (WORA) principle.

### How Java Code Executes

```
  Source Code (.java)
        |
        |   javac (compiler)
        v
  Bytecode (.class)  --------- platform independent
        |
        |   JVM (per operating system / hardware)
```

```
      v
Machine Code  -------------- executed by the OS
```

### Main Components of JVM

| Component | Role |
|---|---|
| Class Loader | Loads .class files into memory; performs Loading, Linking, and Initialization |
| Runtime Data Areas | Memory divided into Method Area, Heap, Stack, PC Registers, and Native Method Stack |
| Execution Engine | Executes bytecode via an Interpreter and a Just-In-Time (JIT) Compiler for performance |
| Native Method Interface (JNI) | Allows Java code to call/be called by native applications and libraries (e.g. C/C++) |
| Garbage Collector | Automatically reclaims memory occupied by objects that are no longer reachable |

### JDK vs JRE vs JVM

| Term | Full Form | Contains | Used For |
|---|---|---|---|
| JVM | Java Virtual Machine | Only the execution engine | Running bytecode |
| JRE | Java Runtime Environment | JVM + core class libraries | Running Java applications |
| JDK | Java Development Kit | JRE + compiler (javac), debugger, dev tools | Developing AND running Java applications |

## 2.9 Unit II — Summary & Practice Questions

### Key Takeaways

- Inheritance lets classes reuse and extend behavior; Java supports single, multilevel, and hierarchical inheritance directly, but not multiple inheritance of classes
- Polymorphism has two forms: compile-time (overloading) and runtime (overriding)
- Interfaces provide multiple inheritance of type and define a pure contract; abstract classes provide partial implementation plus shared state
- Threads allow concurrent execution; implementing Runnable is generally preferred over extending Thread
- The JVM is the cornerstone of Java's platform independence, converting bytecode to machine-specific instructions at runtime

### Practice Questions

1. Differentiate between method overloading and method overriding with a suitable example of each.
2. Why does Java not support multiple inheritance through classes? Explain how interfaces solve this problem.
3. What is an abstract class? Can an abstract class have a constructor? Justify your answer.
4. Explain the life cycle of a thread in Java with a neat diagram.

5. Describe the architecture of the JVM and explain the role of the Class Loader and the Garbage Collector.
6. What is the difference between the Runnable interface and extending the Thread class for creating a thread?
7. Write a Java program to demonstrate runtime polymorphism using a superclass reference pointing to different subclass objects.

## UNIT III

***Development Tools — Eclipse, ADT Plug-in & Android Emulator***

## 3.1 Introduction to the Android Development Environment

Building an Android application requires a coordinated set of tools: an IDE to write code, an SDK containing Android platform libraries, a plug-in that connects the IDE to the Android build system, and either a physical device or a virtual device (emulator) to test the app on. This unit walks through setting up this classic toolchain — Eclipse IDE with the ADT (Android Development Tools) plug-in — historically the primary Android development environment before Android Studio became the official IDE.

> **Note:** Eclipse ADT was officially deprecated by Google in 2015 in favor of Android Studio. It remains an important part of the academic syllabus because it clearly separates each moving part (IDE, plug-in, SDK, AVD) that Android Studio later bundled together — understanding it builds a strong conceptual foundation.

### Components Required

| Component | Purpose |
|---|---|
| JDK (Java Development Kit) | Android apps are written in Java; Eclipse and the Android build tools both require a JDK |
| Eclipse IDE (Classic or IDE for Java Developers) | The base code editor and workspace |
| ADT Plug-in | Integrates Android project templates, the SDK, DDMS, and the emulator controls into Eclipse |
| Android SDK | Contains platform tools, build tools, platform APIs (e.g. Ice Cream Sandwich/Jelly Bean), and system images |
| AVD (Android Virtual Device) / Emulator | Software emulation of an Android device used for testing without physical hardware |

## 3.2 Installing and Using Eclipse with the ADT Plug-in

### Step 1 — Install the JDK

Download and install a compatible JDK (Java SE Development Kit) from Oracle's archive. Set the JAVA_HOME environment variable to the JDK installation directory and add %JAVA_HOME%\bin (Windows) or $JAVA_HOME/bin (Linux/Mac) to the system PATH. Verify with:

```
java -version
javac -version
```

### Step 2 — Install Eclipse IDE

1. Download “Eclipse IDE for Java Developers” (Classic package) from eclipse.org.

2. Extract the downloaded archive to a folder of your choice (Eclipse does not require a traditional installer).
3. Launch eclipse.exe and choose (or create) a workspace folder — this is where all your projects will be stored.

### Step 3 — Install the ADT Plug-in

There are two ways to add ADT to Eclipse:

### Method A — Via Eclipse Update Site (Online)

1. In Eclipse, go to Help → Install New Software…
2. Click Add… and enter a Name (e.g. "ADT Plugin") and the Location URL provided by Google's ADT update site.
3. Select the Developer Tools checkbox (this includes Android DDMS and Android Development Tools) and click Next.
4. Accept the license agreement and click Finish. Eclipse will download and install the plug-in.
5. Restart Eclipse when prompted.

### Method B — Offline / ZIP Installation

1. Download the ADT plug-in ZIP archive.
2. In Eclipse, go to Help → Install New Software… → Add… → Archive… and browse to the downloaded ZIP.
3. Follow the same selection, license-acceptance, and restart steps as Method A.

### Step 4 — Point Eclipse to the Android SDK

1. After restarting, go to Window → Preferences → Android.
2. Browse to and select the folder where the Android SDK is installed (or let Eclipse install it if prompted during first launch).
3. Click Apply and OK. Eclipse will now show an Android SDK Manager icon on the toolbar.

## 3.3 Installing a Virtual Machine for Android (Emulator)

The AVD Manager creates virtual Android devices ("AVDs") that emulate specific Android versions and hardware profiles — such as Android 4.0 Ice Cream Sandwich or Android 4.1–4.3 Jelly Bean — without needing a physical phone or tablet.

### Steps to Create an AVD

1. Open the SDK Manager (via the Eclipse toolbar icon, or Window → Android SDK Manager) and ensure the desired platform's System Image (e.g. Android 4.1 Jelly Bean, API Level 16) is downloaded and installed, alongside the SDK Platform-tools and Build-tools.
2. Open the AVD Manager: Window → Android Virtual Device Manager.
3. Click Create… (or New…) to open the "Create new Android Virtual Device" dialog.
4. Fill in the AVD configuration:
   - AVD Name — e.g. "JellyBean_Test_Device"
   - Device — a hardware profile (e.g. Nexus, or a custom screen size)
   - Target — the platform/API level, e.g. "Android 4.1 – API Level 16" for Jelly Bean
   - CPU/ABI — typically ARM or Intel Atom (x86) — x86 images run noticeably faster on most development PCs

- SD Card size and Skin — optional but recommended for testing storage and different screen resolutions

5. Click OK to save the AVD, then select it in the list and click Start… to launch the emulator and verify it boots correctly.

> **Note:** The first boot of a new AVD can take several minutes. Enabling hardware acceleration (Intel HAXM, on Intel-based development machines) significantly speeds up subsequent emulator launches.

## 3.4 Configuring the Installed Tools

Once Eclipse, ADT, the SDK, and an AVD are all installed, a few configuration checks ensure the full toolchain works together correctly:

- SDK Location — confirm under Window → Preferences → Android that the SDK path is correctly detected (a green confirmation, not a red error, should appear)
- Installed Packages — open the SDK Manager and confirm the target Android Platform, SDK Platform-tools, and SDK Build-tools are all marked as "Installed"
- Environment Variables — confirm JAVA_HOME (and, on some setups, ANDROID_HOME pointing to the SDK folder) are correctly set at the OS level
- Eclipse Project Build Target — each Android project's properties should specify a build target consistent with the installed SDK platform
- USB Drivers — for physical-device testing later, ensure the appropriate OEM USB driver for the device is installed (Windows especially)

## 3.5 Creating an Android Project

1. In Eclipse, go to File → New → Project… and select Android → Android Application Project.
2. Enter the Application Name (e.g. "HelloWorld") — the Project Name and Package Name (e.g. com.example.helloworld) will be suggested automatically and can be edited.
3. Select the Minimum Required SDK, Target SDK, and Compile With versions (choose values compatible with the AVD created earlier, e.g. API 16 for Jelly Bean).
4. Click Next through the configuration screens (Configure Project, choose a Launcher Icon, choose Blank Activity as the activity template).
5. Set the Activity Name (e.g. MainActivity) and the Layout Name (e.g. activity_main), then click Finish.

Eclipse/ADT will generate a complete project skeleton, including the AndroidManifest.xml, a default res/layout/activity_main.xml layout file, and a MainActivity.java class.

## 3.6 Hello World — Running on the Emulator

The default project template generated by Eclipse already displays "Hello world!" via a TextView defined in the layout XML — for the purposes of this exercise, no additional code is strictly required, but the layout and activity are shown below for clarity.

### res/layout/activity_main.xml (generated)

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent" >

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerInParent="true"
        android:text="Hello world!" />
```

```
</RelativeLayout>
```

### src/.../MainActivity.java (generated)

```
package com.example.helloworld;

import android.app.Activity;
import android.os.Bundle;

public class MainActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

### Steps to Run on the Emulator

1. Right-click the project in the Package Explorer → Run As → Android Application.
2. If the previously created AVD is not already running, Eclipse/ADT will prompt the AVD selection dialog — select the AVD (e.g. JellyBean_Test_Device) and click OK.
3. Eclipse builds the project into an APK, launches the emulator (if not already running), installs the APK, and starts the app automatically.
4. Once the emulator finishes booting the OS and installing the app, the “Hello world!” screen appears on the emulator display.

> **Note:** The Console and LogCat views in Eclipse (Window → Show View → Android → LogCat) are essential for viewing build output and runtime logs/errors during this process.

## 3.7 Deploying on a USB-Connected Android Device

Testing on a real device is faster than the emulator and reveals real-world performance and hardware behavior. The steps below assume the device runs the same or a compatible Android version to the project's target SDK.

1. On the Android device, enable Developer Options — go to Settings → About Phone and tap Build Number seven times (on Ice Cream Sandwich/Jelly Bean and later).
2. Inside the newly visible Settings → Developer Options, enable USB Debugging.
3. Connect the device to the development PC using a USB cable.
4. On Windows, ensure the correct OEM USB driver for the device is installed so it is recognized (on Linux/Mac this step is usually unnecessary).
5. Verify the device is detected by the Android debugging bridge (adb) from a command prompt/terminal:

```
adb devices

List of devices attached
XXXXXXXXXXXX    device
```

6. Back in Eclipse, right-click the project → Run As → Android Application.
7. In the device chooser dialog that appears, select the physical device (listed under its device ID) instead of an AVD, and click OK.
8. Eclipse builds and installs the APK directly on the connected device via adb, and the app launches automatically on the device screen.

### Emulator vs Physical Device — Quick Comparison

| Aspect | Emulator (AVD) | USB-Connected Device |
|---|---|---|
| Setup effort | No hardware needed; slower first boot | Requires USB debugging + drivers |
| Performance | Slower, especially on ARM images | Native, real-world performance |
| Hardware features | Limited (sensors/camera often simulated) | Full access to real sensors, camera, GPS |
| Best for | Quick iteration, testing multiple screen sizes/API levels | Final testing, performance profiling, hardware-dependent features |

## 3.8 Unit III — Summary & Practice Questions

### Key Takeaways

- A complete Android toolchain needs a JDK, an IDE (Eclipse), a plug-in bridging the IDE to Android (ADT), the Android SDK, and a way to test (AVD or physical device)
- The ADT plug-in can be installed via Eclipse's online update site or via an offline ZIP archive
- An AVD emulates a specific Android version/hardware profile and must match a downloaded SDK platform system image
- A new Android project auto-generates a manifest, a default layout, and a launcher Activity that already displays “Hello world!”
- Deploying to a physical device requires enabling Developer Options and USB Debugging on the device, plus correct USB drivers on the PC

### Practice Questions

1. List the components required to set up a complete Android development environment using Eclipse, and explain the role of each.
2. Explain the two methods of installing the ADT plug-in into Eclipse.
3. What is an AVD? Describe the steps to create one for testing an Android 4.1 (Jelly Bean) application.
4. Outline the steps to create a new Android Application Project in Eclipse with ADT.
5. What settings must be enabled on a physical Android device before it can be used for USB-based app deployment?
6. Compare testing an application on an emulator versus a physical USB-connected device, listing at least three points of difference.